

Introduction of SQL

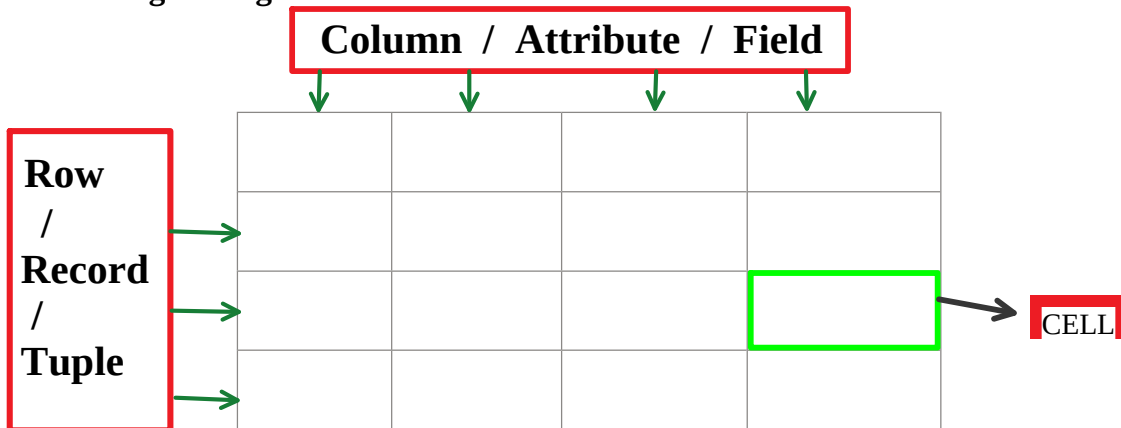
SQL is standard computer language

SQL stands for **STRUCTURED QUERY LANGUAGE**

SQL is used to communicate / interact and manipulate the **Relational DATABASE**(Data will be stored in the format of Table)

Table:

Table is logical organization of Data which consists of Rows and columns



WHAT CAN SQL DO.....?

SQL can execute queries against a database

SQL can retrieve data from a database

SQL can insert records in a database

SQL can update records in a database

SQL can delete records from a database

SQL can create new tables in a database

SQL can create views in a database

DATA BASE

*"Data base is place **or** media which we can store the data in systematic and organized manner"*

○ *The basic operations that can be performed on the database are*

- CREATE / INSERT
- READ / RETRIVE
- UPADTE / MODIFY
- DROP / DELETE



○ *"This operation are universally known as “**CRUD**” operation"*

STATEMENTS

1. DATA QUERY LANGUAGE (DQL)
2. DATA DEFINITION LANGUAGE (DDL)
3. DATA MANIPULATION LANGUAGE (DML)
4. TRANSCATION CONTROL LANGUAGE (TCL)
5. DATA CONTROL LANGUAGE (DCL)

DATA QUERY LANGUAGE (DQL) :

" DQL is used to retrieve/Read/fetch the data from the database " .

SELECT :"It is used to retrieve the *data* from the table and display it.

Possibilities of Select statement :

1. PROJECTION
2. SELECTION
3. JOIN

PROJECTION :

"It is a process of retrieving the data by **selecting only the columns** is known as Projection "

- In projection all the records / values present in a particular columns are by default selected .

SYNTAX :

SELECT * / **[DISTINCT]** Column_Name / Expression **[ALIAS]**
FROM Table_Name ;

NOTE

- Asterisk(*): it means to select all the columns from the table .
- Semicolon: it means end of the query .

ORDER OF EXECUTION

- 1 FROM Clause (here, only we can pass Table Name as an argument .)
- 2 SELECT Clause

DISTINCT Clause

" It is used to remove the duplicate or repeated values from the Result table ".

Example :

SELECT DISTNICT SNAME,PER,BRANCH
FROM STUDENT;

Student

<u>SID</u>	<u>SNAME</u>	<u>BRANCH</u>	<u>PER</u>
1	A	ECE	60
2	B	CSE	75
3	C	ME	50
4	D	ECE	80
5	C	CSE	50
6	E	CIVIL	95

SNAME	PER	BRANCH
A	60	ECE
B	75	CSE
C	50	ME
D	80	ECE
C	50	CSE
E	95	CIV

SNAME	PER	BRANCH
A	60	ECE
B	75	CSE
C	50	ME
C	50	CSE
D	80	ECE
E	95	CIV

SELECTION :

"It is a process of retrieving the data by *selecting both the columns and rows* is known as Selection " .

SELECT * / [DISTINCT] Column_Name / Expression [ALIAS]
FROM Table_Name
WHERE <Filter_Condition> ;

ORDER OF EXECUTION

1. FROM
2. WHERE ("Where clause is used to filter the records ".)
3. SELECT

GROUP-BY Clause

Group by clause is used to group the records .

SYNTAX:

```
SELECT group_by_expression / group_function  
FROM table_name  
[WHERE <filter_condition>]  
GROUP BY column_name/expression
```

ORDER OF EXECUTION:

1. FROM
2. WHERE (if used) [ROW-BY-ROW]
3. GROUP BY [ROW-BY-ROW]
4. SELECT [GROUP-BY-GROUP]

EMP

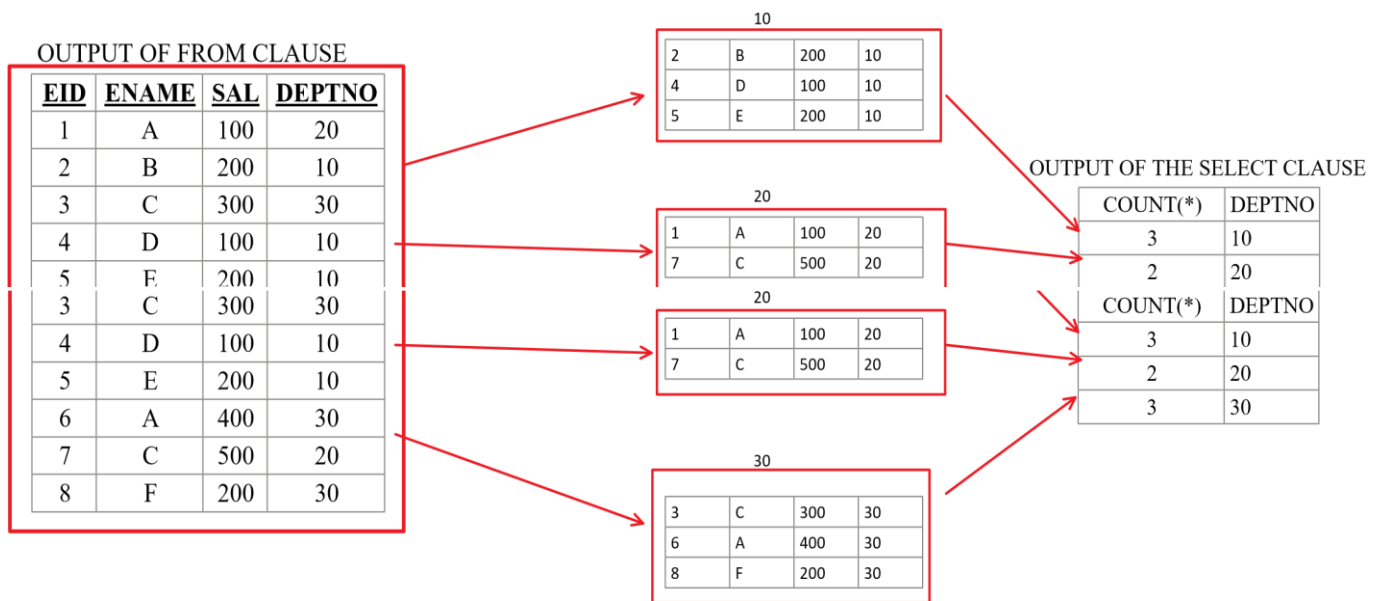
<u>EID</u>	<u>ENAME</u>	<u>SAL</u>	<u>DEPTNO</u>
1	A	100	20
2	B	200	10
3	C	300	30
4	D	100	10
5	E	200	10
6	A	400	30
7	C	500	20
8	F	200	30

Example :

WAQTD number of employees working in each dept .

```
SELECT COUNT(*),DEPTNO  
FROM EMP  
GROUP BY DEPTNO;
```

OUTPUT OF GROUP BY CLAUSE



HAVING Clause

" Having Clause is used to Filter the Group "

SYNTAX:

```

SELECT group_by_expression / group_function
FROM table_name
[WHERE <filter_condition>]
GROUP BY column_name/expression
HAVING <group_filter_condition>
    
```

ORDER OF EXECUTION:

1. FROM
2. WHERE(if used) [ROW-BY-ROW]
3. GROUP BY(if used) [ROW-BY-ROW]
4. HAVING (if used) [GROUP-BY-GROUP]
5. SELECT [GROUP-BY-GROUP]

Example :

WAQTD to find number of employees working in each Dept if there are at least 3 employees in each dept .

```

SELECT DEPTNO ,COUNT(*)
FROM EMP
GROUP BY DEPTNO
HAVING COUNT(*)>=3;
    
```

OUTPUT OF GROUP BY CLAUSE

OUTPUT OF FROM CLAUSE

EID	ENAME	SAL	DEPTNO
1	A	100	20
2	B	200	10
3	C	300	30
4	D	100	10
5	E	200	10
6	A	400	30
7	C	500	20
8	F	200	30

10

2	B	200	10
4	D	100	10
5	E	200	10

20

1	A	100	20
7	C	500	20

30

3	C	300	30
6	A	400	30
8	F	200	30

OUTPUT OF HAVING CLAUSE GROUP_FILTER CONDITON

2	B	200	10
4	D	100	10
5	E	200	10

DEPTNO 30

3	C	300	30
6	A	400	30
8	F	200	30

OUTPUT OF SELECT CLAUSE

DEPTNO	COUNT(*)
10	3
30	3

COUNT(*)>=3
DEPTNO 10

3>=3

2>=3

3>=3

ORDER BY CLAUSE

Order by clause is used to arrange the records either in ascending order or descending order

SYNTAX:

```
SELECT group_by_expression / group_function  
FROM table_name  
[WHERE <filter_condition>]  
[GROUP BY column_name/expression]  
[HAVING <group_filter_condition>]  
ORDER BY Column_name/expression [ASC]/DESC
```

ORDER OF EXECUTION:

1. FROM
2. WHERE(if used) [ROW-BY-ROW]
3. GROUP BY(if used) [ROW-BY-ROW]
4. HAVING (if used) [GROUP-BY-GROUP]
5. SELECT [GROUP-BY-GROUP]
- 6 ORDER BY [ROW- BY-ROW]

EXAMPLE

```
SELECT SAL  
FROM EMP  
ORDER BY SAL [ASC];
```

SAL
120
300
50

SAL
50
120
300

```
SELECT SAL  
FROM EMP  
ORDER BY SAL DESC
```

SAL
300
120
50

OPERATORS IN SQL

- 1) ARITHMETIC OPERATORS :- (+ , - , * , /)
- 2) CONCATENATION OPERATOR :- (||)
- 3) COMPARISON OPERATORS :- (= , != or <>)
- 4) RELATIONAL OPERATOR :- (> , < , >= , <=)
- 5) LOGICAL OP : (AND , OR , NOT)
- 6) SPECIAL OPERATOR :-
 1. IN (Column_Name / Exp IN (v1 , v2 , . . vn))
 1. NOT IN (Column_Name / Exp NOT IN (v1 , v2 , . . vn))
 2. BETWEEN (Column_Name BETWEEN Lower_Range AND Higher_Range)
 3. NOT BETWEEN (Column_Name NOT BETWEEN Lower_Range AND Higher_Range)
 4. IS (Column_Name IS NULL)
 5. IS NOT (Column_Name IS NOT NULL)
 6. LIKE (Column_Name/ exp LIKE 'pattern')
 7. NOT LIKE
- 7) SUBQUERY OPERATORS:
 1. ALL
 1. ANY
 2. EXISTS
 3. NOT EXISTS
- 8) SET OPERATOR
 1. UNION (it will combine the output by considering common and uncommon values – duplicates will not be present)
 2. UNION ALL (it will combine the output by considering common and uncommon values – duplicates will be present)
 3. INTERSECT (it will combine the output by considering only common values)
 4. MINUS (it will combine the output by considering only uncommon values)

$Q1 = \{1, 2, 3, 4\}$

$Q2 = \{2, 4, 5, 6\}$

$Q1 \text{ union } Q2 = \{1, 2, 3, 4, 5, 6\}$

$Q1 \text{ union all } Q2 = \{1, 2, 3, 4, 2, 4, 5, 6\}$

$Q1 \text{ minus } Q2 = \{1, 3\}$

$Q2 \text{ minus } Q1 = \{5, 6\}$

$Q1 \text{ intersect } Q2 = \{2, 4\}$

Syntax:

```
SELECT COL_NAME  
FROM TABLE_NAME1  
[WHERE <FILTER CODITION>]
```

UNION/ UNION ALL/ INTERSECT/ MINUS

```
SELECT COL_NAME  
FROM TABLE_NAME2  
[WHERE <FILTER CODITION>];
```